

Interaktiver Debugger für Python

Der eingebaute Debugger erlaubt es, Python-Code in Notebooks Schritt für Schritt zu durchlaufen, Haltepunkte zu setzen und Variablenwerte live zu prüfen – ähnlich wie in einer klassischen Entwicklungsumgebung (IDE). Das erleichtert die Fehlersuche erheblich gegenüber dem bloßen Einfügen von Ausgabe-Befehlen.

Systemvoraussetzungen

- Serverless Compute, oder
- Standard-Zugriffsmodus-Cluster mit Databricks Runtime 14.3 LTS oder neuer, oder
- Dedicated-Zugriffsmodus-Cluster mit Databricks Runtime 13.3 LTS oder neuer, oder
- No-Isolation-Shared-Zugriffsmodus-Cluster mit Databricks Runtime 13.3 LTS oder neuer.

Der Debugger muss zusätzlich aktiviert werden: **Settings** → **Developer** → **Python Notebook Interactive Debugger**.

Debug-Sitzung starten

Drei Wege stehen offen:

1. **Run** → **Debug cell** im Menü wählen
2. Tastenkürzel **Alt+Umschalt+D**
3. "Debug cell" im Ausführungsmenü der Zelle auswählen

Tritt beim normalen Ausführen ein Fehler auf, erscheint zusätzlich ein Debug-Button direkt in der Fehlerausgabe, mit dem sich sofort ab der Fehlerstelle debuggen lässt.

Haltepunkte (Breakpoints)

Ein Klick in den Rand (Gutter) links neben einer Codezeile setzt einen Haltepunkt, ein erneuter Klick entfernt ihn. Wichtig: Der Code stoppt **vor** der Ausführung der markierten Zeile, nicht danach – man sieht also den Zustand, bevor diese Zeile wirkt.

Die Debug-Werkzeugleiste

Während einer Debug-Sitzung stehen folgende Aktionen zur Verfügung:

- Haltepunkte setzen/entfernen
- Variablenwerte am aktuellen Haltepunkt einsehen
- Zeile für Zeile durch den Code gehen (Step)
- In Funktionen hinein- oder aus ihnen heraus springen (Step into / Step out)
- Die aktuell ausgeführte Zeile wird im Code farblich hervorgehoben

Variablen-Explorer

Zeigt aktuelle Variablenwerte an und aktualisiert sich automatisch während der Ausführung (ab Databricks Runtime 12.2 LTS). Über "Inspect" lässt sich der Wert einer Variable explizit ausgeben. Lokale Variablen innerhalb von Funktionen sind mit "[local]" markiert. Eine Suchleiste ermöglicht das gezielte Filtern der angezeigten Variablen.

Die Debug-Konsole

Erscheint automatisch im unteren Bereich während eines Haltepunkts und erlaubt die Ausführung von Python-Code zur Inspektion oder Manipulation von Variablen. Mit **Enter** wird der Code ausgeführt, mit **Umschalt+Enter** wird eine neue Zeile innerhalb eines mehrzeiligen Ausdrucks eingefügt. Ausführungen in der Konsole laufen nach 15 Sekunden in ein Timeout; der Befehl `display` wird dort nicht unterstützt.

Sprung in Workspace-Dateien

Sind Tabs für Notebooks und Dateien aktiviert, kann der Debugger auch in Funktionen springen, die in anderen Workspace-Dateien definiert sind – die referenzierte Datei öffnet sich dann in einem neuen Tab. Wichtige Einschränkung: Der Debugger kann nur in Funktionen springen, die in Workspace-Dateien definiert sind. Ein Sprung in Python-Bibliotheken oder andere Notebooks wird noch nicht unterstützt.

Debug-Sitzung beenden

Eine Sitzung endet durch:

- Klick auf den Debugger-Stopp-Button (oben links an der Zelle)
- Klick auf den Notebook-Stopp-Button
- Automatisch nach 30 Minuten Inaktivität

Zusatz: Genie Code

Genie Code kann ergänzend zur Fehlerdiagnose und zum Vorschlagen von Korrekturen genutzt werden.

Wichtige Einschränkung

Werden externe Module nach dem Import geändert, spiegeln sich diese Änderungen beim Debuggen nicht automatisch wider – außer bei aktiviertem "Autoreload" (verfügbar ab Databricks Runtime 18.0 bzw. Serverless-Compute-Version 4).

Quelle: <https://docs.databricks.com/aws/en/notebooks/debugger>

Stand: 2026-08-06